

Progetto

Progettazione di Sistemi Digitali

Circuito di filtraggio convoluzionale di immagini

M. De Fusco, C. Ferrari, L. Lo Bianco, S. Scarcelli



Indice

1. Introduzione e Architettura
2. Componenti Base
3. Unit Testing: Ripple Carry Adder
4. Circuito moltiplicatore
5. Carry Save Adder tree
6. Buffer & Pipeline
7. FSM e AXI-Stream
8. Risultati e Conclusioni

1. Introduzione e Architettura

Analisi preliminari e suddivisione dei lavori

Descrizione del Progetto

Specifiche

Progettazione di un filtro per immagini.

- Immagine: almeno 32x32 pixel a 8 bit (unsigned)
- Filtro: 3x3 con coefficienti 4 bit (signed)
- Anchor point: centrale
- Kernel: *a scelta*

Obiettivo

- Implementazione descrizione *VHDL* del circuito
- Creazione di **testbench** per verificare il *corretto funzionamento*
- Analisi dei report *post implementation*

Architettura generale

Architettura parallela in pipeline

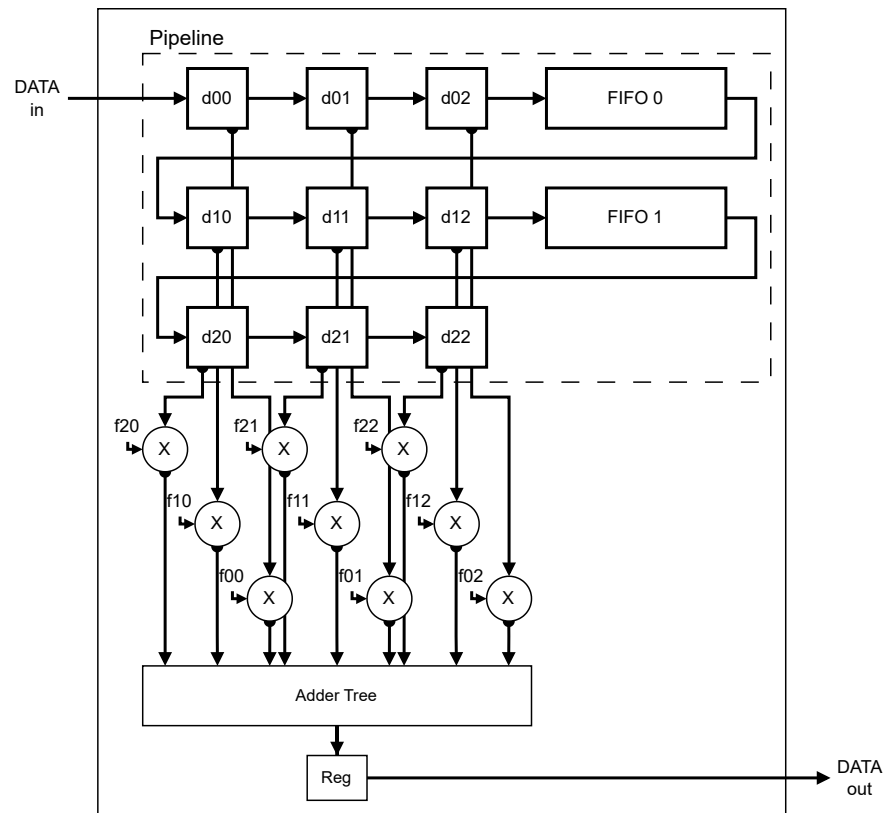
I dati relativi ai pixel dell'immagine sono caricati in pipeline, una volta che il primo pixel dell'immagine raggiunge la posizione centrale (d11), il circuito inizia a elaborare i pixel filtrati ad ogni ciclo di clock.

Timing

- Latenza: 34

$$\left\lfloor \frac{N_{row, kernel}}{2} \right\rfloor \cdot N_{col, img} + \left\lceil \frac{N_{col, kernel}}{2} \right\rceil = \left\lfloor \frac{3}{2} \right\rfloor \cdot 32 + \left\lceil \frac{3}{2} \right\rceil = 34$$

- Throughput: 1



Scelte implementative

Moltiplicatori di Booth e Carry Save Adder tree

Componenti di calcolo aritmetico

Oltre alla *pipeline*, i due componenti principali sono i circuiti moltiplicatori e sommatore implementati rispettivamente:

- Booth Multiplier: moltiplicatore con *pre-computazione* e selezione tramite codifica di Booth
- Carry Save Adder tree: albero di somma Carry Save con Carry Select finale

Circuito di controllo

Per gestire la *pipeline* e le due interfacce AXI-Stream di ingresso e uscita abbiamo implementato una Finite State Machine che:

1. Gestisce l'avanzamento della *pipeline*
2. Gestisce i segnali relativi alle interfacce AXI-Stream di ingresso e uscita

Suddivisione del Lavoro

Membro del Team	Responsabilità Principali
M. De Fusco	Componenti base, Buffer Line e relative <i>testbench</i>
C. Ferrari	Booth Multiplier , relative <i>testbench</i> e script di verifica MATLAB
L. Lo Bianco	Carry Save Adder tree , relative <i>testbench</i> e report
S. Scarcelli	Finite State Machine , relative <i>testbench</i> e <i>testbench</i> complessiva

Anche se i lavori sono stati svolti singolarmente, ognuno ha contribuito a *miglioramenti* o *modifiche* di ogni parte del progetto.

2. Componenti Base

Componenti base e di ausilio

Full Adder

L'elemento atomico per le operazioni aritmetiche.

Implementa la somma di tre bit (A, B, C_{in}).

Logica Implementativa

Abbiamo scelto una descrizione **behavioral** ottimizzata per l'inferenza hardware.

- Somma (S): XOR a 3 ingressi.
- Riporto (C_{out}): Calcolato sfruttando il segnale di propagazione P .
 - Se $P = 1$ ($A \neq B$), il riporto dipende da C_{in} .
 - Se $P = 0$ ($A = B$), il riporto è uguale ad A .

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

```
entity full_adder is
    port (
        a, b, cin : in  STD_LOGIC;
        s, cout    : out STD_LOGIC
    );
end entity full_adder;

architecture behavioral of full_adder is
    signal p : STD_LOGIC;
begin
    -- Calcolo Propagate
    p    <= a xor b;
    s    <= p xor cin;

    -- Mux-based Carry logic (Efficiente su FPGA)
    cout <= cin when p = '1' else a;
end architecture behavioral;
```

N-Bit Ripple Carry Adder (RCA)

Per sommare vettori di bit, colleghiamo Full Adder in cascata

Architettura Parametrica

Il componente è reso generico tramite il parametro `N`. Utilizziamo il costrutto `generate` per istanziare dinamicamente la catena di addizionatori.

- **Vantaggi:** Semplicità e Area ridotta
- **Svantaggi:** Ritardo di propagazione lineare (il riporto deve attraversare tutta la catena)

```
entity ripple_carry_adder is
    generic ( N : POSITIVE ); -- Parametro Generico
    port ( ... );
end entity ripple_carry_adder;

-- Architecture
begin
    c(0) ← cin; -- Carry in iniziale

    loop_n: for i in 0 to N - 1 generate
        fa_i: component full_adder
            port map (
                a    ⇒ a(i),
                b    ⇒ b(i),
                cin  ⇒ c(i),
                s    ⇒ s(i),
                cout ⇒ c(i + 1) -- Chain connection
            );
        end generate loop_n;

    cout ← c(N); -- Ultimo riporto
```

Unit Testing: Full Adder

Verifica esaustiva (Exhaustive testing)

Poiché il Full Adder ha solo 3 ingressi (A, B, C_{in}), lo spazio degli stati è piccolo ($2^3 = 8$ casi). Abbiamo implementato una testbench che itera su una Tabella della Verità completa.

```
type truth_table_type is array (0 to 7)
  of STD_LOGIC_VECTOR(4 downto 0);

constant TRUTH_TABLE : truth_table_type := (
  "000" & "00", -- 0+0+0 = 0 (C=0)
  "011" & "10", -- 0+1+1 = 0 (C=1)
  "111" & "11", -- 1+1+1 = 1 (C=1)
  ...
);

-- Loop di simulazione
for i in 0 to 7 loop
  gen_input <- unsigned(TRUTH_TABLE(i)(4 downto 2));

  -- Testing automatizzato ...
end loop;
```

Punti Chiave

1. **Copertura 100%:** Ogni possibile combinazione di ingresso viene testata
2. **Self-Checking:** L'istruzione `assert` verifica automaticamente l'uscita rispetto all'atteso
3. **Report:** In caso di errore, la simulazione stampa i valori esatti che hanno fallito

Unit Testing: Ripple Carry Adder

Verifica mirata (Directed testing)

Per l'addizionatore a N bit, testare tutte le combinazioni richiederebbe troppo tempo (casi). Abbiamo optato per una **Lookup Table** contenente casi specifici (*Corner Cases*).

```
constant TRUTH_TABLE : truth_table_type := (  
  -- Caso Base: 5 + 5 = 10  
  2 => std_logic_vector(x"05") &  
    std_logic_vector(x"05") &  
    '0' &  
    ...  
  
  -- Caso Overflow (Carry Out generation):  
  -- 255 (FF) + 1 (01) = 0 (00) con Carry=1  
  3 => std_logic_vector(x"FF") &  
    std_logic_vector(x"01") &  
    '0' & -- Cin  
    '1' & -- Expected Cout  
    std_logic_vector(x"00"), -- Expected Sum  
  
  -- Caso Saturazione (Max Propagation):  
  -- 255 (FF) + 255 (FF) + 1 = 255 (FF) con Carry=1
```

Scenari Testati

Il vettore di test copre le criticità del ripple carry:

1. **Zero + Zero:** Verifica reset/base
2. **Somma standard:** Verifica funzionalità aritmetica
3. **Overflow (8-bit):** Verifica che il riporto esca correttamente dal MSB ($C_{out} = 1$)
4. **Propagazione Completa:** Verifica il ritardo e la correttezza quando il riporto deve attraversare tutta la catena ($C_{in} \rightarrow C_{out}$)

3. Circuito moltiplicatore

Strategia con codifica *Booth* a 4-bit

Strategia di Implementazione

A differenza di un moltiplicatore di Booth standard che raggruppa i bit, la nostra implementazione sfrutta la dimensione ridotta dei coefficienti del kernel (3×3 , 4-bit signed)

Hard-Coded Coefficients

Poiché un numero a 4 bit ha solo $2^4 = 16$ combinazioni possibili (da -8 a +7), abbiamo codificato staticamente il comportamento per ogni caso.

- **Input:** Pixel (P) a 8-bit (Unsigned), Coefficiente (F) a 4-bit (Signed).
- **Output:** Prodotto (M) a 12-bit (Signed).
- **Ottimizzazione:** Sostituzione delle moltiplicazioni costose con operazioni di **Shift** e **Somma/Sottrazione**.

Codice VHDL

Il modulo calcola in parallelo i 9 prodotti necessari per la convoluzione

- Porte P: 9 Pixel della finestra corrente (8-bit)
- Porte F: 9 Coefficienti del filtro (4-bit)
- Porte M: 9 Risultati parziali (12-bit)

La dimensione di uscita è garantita per evitare overflow:

$$\begin{aligned} Dim_{out} &= Dim_{img} + Dim_{coeff} = \\ &= 8 + 4 = 12 \text{ bit} \end{aligned}$$

```
entity booth_multiplier is
  generic(
    componente_immagine : POSITIVE := 8;
    coefficiente_filtro   : POSITIVE := 4;
    somma                : POSITIVE := 12
  );
  port (
    -- Input Pixel (3x3 Matrix)
    P_1_1, ..., P_3_3 : in std_logic_vector(7 downto 0);

    -- Input Coefficients (Fixed/Configurable)
    F_1_1, ..., F_3_3 : in std_logic_vector(3 downto 0);

    -- Output Products
    M_1_1, ..., M_3_3 : out std_logic_vector(11 downto 0)
  );
end entity booth_multiplier;
```

Decomposizione del Prodotto

Per semplificare l'hardware, ogni moltiplicazione $P \times F$ viene scomposta in due sottoprodotti parziali (PP_A e PP_B) facili da calcolare tramite shift

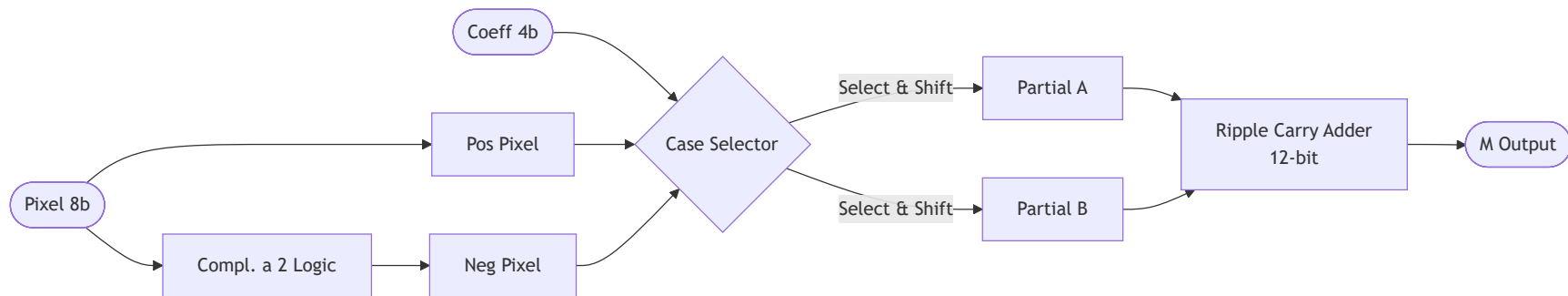
$$R = (P \times A) + (P \times B)$$

Coeff (Bin)	Val (Dec)	A	B	Operazione Hardware
0000	0	0	0	0
0001	1	1	0	P
0010	2	-2	4	(NOT(P)+1)<<1 + P<<2
0011	3	-1	4	(NOT(P)+1) + P<<2
0100	4	0	4	P<<2
0101	5	1	4	P + P<<2
0110	6	-2	8	(NOT(P)+1)<<1 + P<<3
0111	7	-1	8	(NOT(P)+1) + P<<3

Coeff (Bin)	Val (Dec)	A	B	Operazione Hardware
1000	-8	0	-8	(NOT(P)+1)<<3
1001	-7	1	-8	P + (NOT(P)+1)<<3
1010	-6	-2	-4	(NOT(P)+1)<<1 + (NOT(P)+1)<<2
1011	-5	-1	-4	(NOT(P)+1) + (NOT(P)+1)<<2
1100	-4	0	-4	(NOT(P)+1)<<2
1101	-3	1	-4	P + (NOT(P)+1)<<2
1110	-2	-2	0	(NOT(P)+1)<<1
1111	-1	-1	0	(NOT(P)+1)

Architettura interna del Moltiplicatore

Schema logico per un singolo pixel (replicato 9 volte)



- Selettore: Un CASE statement
- RCA Finale: Somma i due termini parziali estesi a 12 bit per ottenere il risultato finale

Testing e Validazione

Abbiamo verificato il moltiplicatore coprendo i casi limite (*Corner Cases*) e spazzolando tutto il range dei coefficienti

Casi di Test

1. Max Positivo: Pixel 255 Coeff 7
 - Atteso: 1785
2. Max Negativo: Pixel 255 Coeff -8
 - Atteso: -2040
3. Coefficient Sweep:
 - Input fisso a 255.
 - Loop coefficienti da -8 a +7.

```
-- Test Max Positive
p ← std_logic_vector(to_unsigned(255, 8));
f ← std_logic_vector(to_signed(7, 4));

-- Test Sweep Range
p ← std_logic_vector(to_unsigned(255, 8));
for i in -8 to 7 loop
    f ← std_logic_vector(to_signed(i, 4));
    wait for 50 ns;
end loop;
```

3. Carry Save Adder tree

Somma parallela ad alte prestazioni

Strategia di Somma: Perché Carry Save?

Dobbiamo sommare 9 operandi da 12-bit (i prodotti parziali) in un singolo ciclo di clock

Il Problema del Ripple Carry

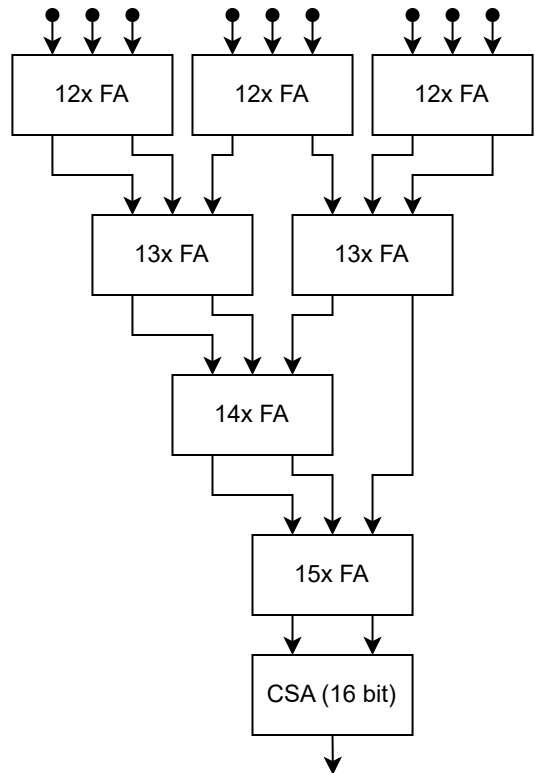
Collegare 8 sommatori classici in cascata creerebbe una catena di riporto lunghissima (Critical Path elevato).

$$T_{delay} \approx 9 \times T_{adder}$$

La Soluzione CSA

L'approccio Carry Save spezza la catena del riporto.

- Si sommano gli operandi a gruppi di 3
- Invece di un risultato unico, si generano due vettori:
 - Sum e Carry
- Il riporto non si propaga orizzontalmente, ma viene salvato per il livello successivo



Codice VHDL

L'entity riceve i 9 prodotti parziali dai moltiplicatori

```
entity carry_save_adder_tree is
  generic (N : POSITIVE := 12);
  port (
    -- 9 Input da 12-bit (Signed)
    i_1_1, i_1_2, i_1_3 : in std_logic_vector(N-1 downto 0);
    i_2_1, i_2_2, i_2_3 : in std_logic_vector(N-1 downto 0);
    i_3_1, i_3_2, i_3_3 : in std_logic_vector(N-1 downto 0);

    -- Output finale esteso a 16-bit per evitare overflow
    sum : out std_logic_vector(N+3 downto 0)
  );
end carry_save_adder_tree;
```

Bit Growth: Da 12 bit in ingresso passiamo a 16 bit in uscita per accomodare la somma massima teorica:

$$MaxVal \approx 9 \times 2^{12} \rightarrow \text{Requires 16 bits}$$

Gestione dei livelli e allineamento bit

La sfida principale è stata la gestione manuale delle estensioni di segno e degli shift tra i vari livelli dell'albero

Livelli FA

I 9 input vengono raggruppati in 3 blocchi da 3. Ogni blocco genera *Sum* e *Carry*.

Regole di Cablaggio:

1. **Vettore Carry:** Shift a sinistra di 1 (peso aritmetico $\times 2$) + riempimento LSB con '0'.
2. **Vettore Sum:** Estensione del segno (MSB) per pareggiare la lunghezza.

Questo processo si ripete, allargando i vettori ($12 \rightarrow 13 \rightarrow 14 \rightarrow 15$ bit) fino ad avere solo due vettori finali da 16 bit.

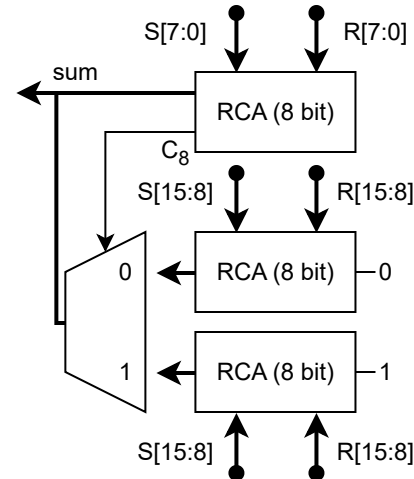
Stadio Finale: Carry Select Adder

Arrivati a due soli vettori (*Somma Parziale e Riporti Parziali*), serve un vero addizionatore finale

Scelta Architeturale

Abbiamo optato per un Carry Select Adder (CSLA) semplificato.

- **Compromesso:** Più veloce di un Ripple Carry, meno risorse di un Kogge-Stone
- **Semplificazione:** Non gestiamo il finale e il relativo MUX, poiché il range a 16-bit copre garantitamente il caso peggiore (Overflow impossibile per design)



Verifica: Worst Case Analysis

Il modulo è stato validato simulando i casi limite matematici

Max Positivo (+16065)

- Pixel: $9 \times (+255)$
- Coeff: $9 \times (+7)$
- Op: $9 \times (255 \times 7) = 9 \times 1785 = 16065$
- Binario: 0011 1110 1100 0001

Max Negativo (-18360)

- Pixel: $9 \times (+255)$
- Coeff: $9 \times (-8)$
- Op: $9 \times (255 \times -8) = 9 \times -2040 = -18360$
- Binario: 1011 1000 0100 1000

Risultato Testbench

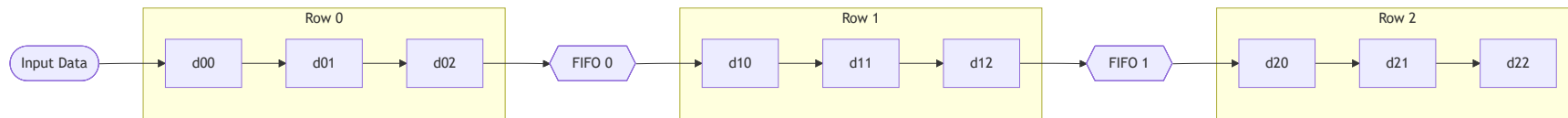
La simulazione conferma che l'albero di somma propaga correttamente i bit di segno e gestisce l'aritmetica in complemento a due senza errori.

5. Buffer & Pipeline

Registri di finestra e FIFO per pipeline

Architettura Sliding Window

Il modulo `buffer_line` trasforma il flusso seriale di pixel in una matrice 3x3 accessibile in parallelo in un singolo ciclo di clock



I dati viaggiano dalla porta AXI-Stream di ingresso verso l'ultima posizione, caricando l'immagine in ordine inverso nei registri di finestra (`d22` pixel precedente a `d00`).

Implementazione: Shift Register Arrays

Le FIFO sono implementate usando una struttura parametrica

Viene definito un `array` di `std_logic_vector` con un numero di elementi pari a $N_{col,img} - N_{col,ker}$.

```
-- Calcolo dimensione buffer:
-- Image Width (32) - Window Size (3) = 29 elementi di buffer
type reg_array is array (ncol-4 downto 0) of std_logic_vector(7 downto 0);
signal buffer1, buffer2 : reg_array;

-- Processo di scorrimento (Line Buffer 1)
process(clk)
begin
    if rising_edge(clk) then
        if valid = '1' then
            -- Ingresso dal registro finale della riga precedente
            buffer1(0) <= d02s;
            -- Shift dell'intera riga
            for j in 1 to ncol-4 loop
                buffer1(j) <= buffer1(j-1);
            end loop;
        end if;
    end if;
end process;
```

Gestione del Flush

Quando l'ultimo pixel dell'immagine entra nella pipeline, la finestra non è ancora vuota (l'ultimo pixel non è ancora stato filtrato)

Logica di "Padding" a Zeri

Quando il segnale `flush` è attivo, l'ingresso viene forzato a `'0'`, permettendo ai pixel reali di avanzare nei registri successivi senza introdurre artefatti.

```
if valid = '1' then
  -- Se siamo in fase di FLUSH, carichiamo zeri (Padding)
  -- per svuotare la pipeline dai pixel validi residui
  if flush = '1' then
    d00s ← (others ⇒ '0');
  else
    d00s ← data; -- Pixel reale
  end if;

  d01s ← d00s; -- Shift...
end if;
```

Questo garantisce che l'ultimo pixel dell'immagine venga processato correttamente al centro della finestra (`d11`) prima di chiudere il frame.

6. Finite State Machine e Interfacce AXI-Stream

Orchestrazione della *pipeline* e interfacce AXI-Stream

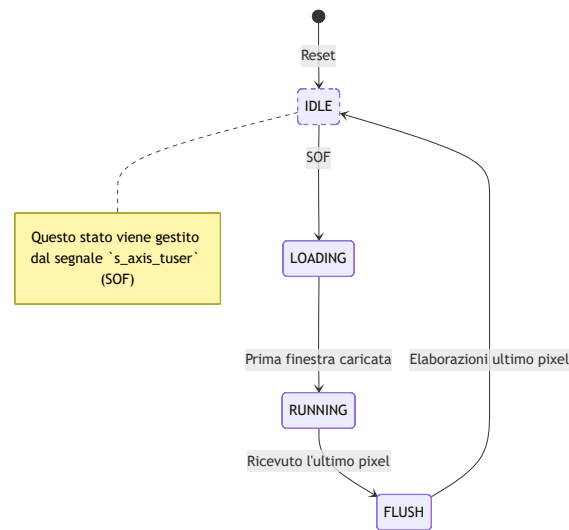
Diagramma degli Stati della FSM

La FSM è strutturata nel seguente modo

La macchina a stati principale controlla il *riempimento* e lo *svuotamento* della finestra 3×3 .

Ogni stato gestisce 3 segnali relativi alla pipeline:

- `pipeline_en` : gestisce l'avanzamento dei dati nella pipeline
- `window_valid` : gestisce il registro di uscita (sampling del pixel filtrato)
- `flush_pipeline` : gestisce se leggere un nuovo pixel o *zero* in ingresso alla pipeline



Nota: Lo stato di IDLE è assente in quanto si fa uso del segnale `s_axis_tuser` (Start Of Frame) per avviare la *pipeline*.

Gestione dell'Interfaccia AXI Stream

Il design agisce come un nodo di elaborazione stream (slave in ingresso, master in uscita), gestendo *handshake*, i segnali `TLAST` e `TUSER` e gestendo eventuale *back-pressure* a valle del circuito.

AXI Slave (Ingresso)

- Handshake: `s_axis_tready` viene asserito quando la pipeline può avanzare (non siamo in `FLUSH` e l'uscita non è bloccata)
- SOF (`tuser`): Campionato per avviare la pipeline
- EOL (`tlast`): Campionato per contare le righe in ingresso e innescare la fase di `FLUSH` al caricamento dell'ultimo pixel

AXI Master (Uscita)

- Handshake: `m_axis_tvalid` è alto *solo* quando `window_valid = '1'` (finestra 3x3 piena e dato in uscita valido)
- SOF (`tuser`): Rigenerato artificialmente e tenuto alto per un solo ciclo in corrispondenza del primo pixel valido in uscita
- EOL (`tlast`): Generato internamente da un contatore di *colonna* per segnalare la fine della riga elaborata

Start of Frame (SOF) & Rimozione dell'IDLE

Sfruttamento del segnale `tuser` per eliminare lo stato di `IDLE`

Invece di avere uno stato inattivo la FSM si ferma fino a quando non arriva un SOF.

```
-- Processo Sincrono della FSM
if rising_edge(s_axis_clk) then
    if s_axis_rstn = '0' then
        current_state ← LOADING;
        -- Reset registri...

    elsif pipeline_en_s = '1' then

        -- LOGICA DI SINCRONIZZAZIONE SOF
        -- Se arriva un TUSER valido, è SEMPRE l'inizio di un nuovo frame.
        if s_axis_tvalid = '1' and s_axis_tuser = '1' then
            current_state ← LOADING;
            latency_counter ← 1; -- Abbiamo già il primo pixel
            flush_pipeline_s ← '0'; -- Interrompe flush precedenti
            m_axis_tuser_s ← '0';
            -- Reset contatori...
        else
            -- Evoluzione normale degli stati (LOADING, RUNNING, FLUSH)
```


Controllo Pipeline e Backpressure

La logica combinatoria che governa l'abilitazione dei registri (`pipeline_en`) è il cuore del dataflow

```
-- 1. Controllo Pipeline (Avanzamento Globale)
pipeline_en_s ← '1' when ((s_axis_tvalid = '1' or current_state = FLUSH)
                        and (m_axis_tready = '1' or window_valid_s = '0'))
                        else '0';

-- 2. Handshake Ingresso (TREADY)
s_axis_tready ← '1' when (current_state ≠ FLUSH and m_axis_tready = '1') else '0';
```

Analisi della Backpressure

Il segnale `m_axis_tready` in ingresso indica se il ricevitore a valle può accettare dati. Se va basso (`'0'`) mentre la finestra è valida, `pipeline_en_s` va a `'0'` . L'intero datapath si "*congela*", preservando i risultati parziali senza perdere alcun pixel, per poi ripartire istantaneamente appena il ricevitore torna pronto.

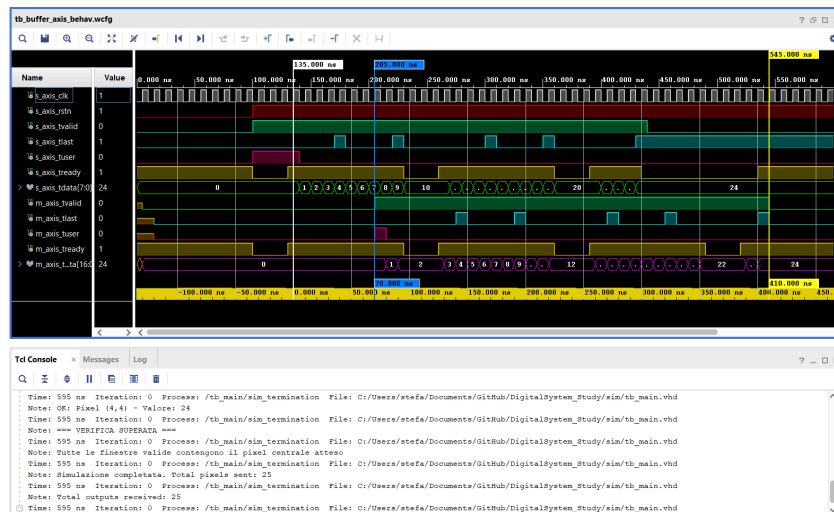
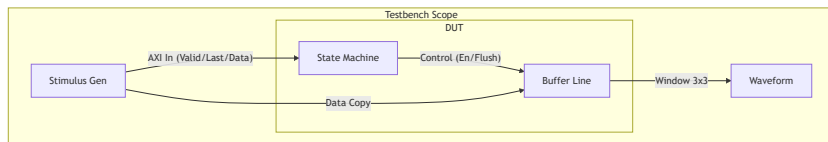
Strategia di verifica di funzionamento

Abbiamo creato una testbench per verificare il corretto funzionamento della FSM insieme alla `buffer_line`

Setup di Simulazione

Per facilitare il debug visivo delle waveform, abbiamo ridotto i parametri rispetto al design reale:

- Immagine: 5x5 pixel (invece di 32x32)
- Pattern Dati: Incrementale
 - *Pixel: $Row \times Width + Col$ (0, 1, 2, ...)*
 - *Nessuna elaborazione*
- Obiettivo: Verificare i casi limite (Start of Frame, End of Line, Flush finale)



Generazione Stimoli e Handshake

Il processo di stimolo emula il flusso dati AXI-Stream in ingresso e legge il flusso dati AXI-Stream in uscita

Inoltre viene simulata la *backpressure* e l'esecuzione del *flush* anche in non validità dell'ingresso per testare la robustezza della FSM.

```
-- Generazione Immagine 5x5
for row in 0 to NROW-1 loop
  for col in 0 to NCOL-1 loop
    -- Data = Indice univoco
    s_axis_tdata ← std_logic_vector( ... );
    s_axis_tvalid ← '1';

    -- Wait for Handshake (Backpressure)
    wait until rising_edge(clk);
    while s_axis_tready = '0' loop
      wait until rising_edge(clk);
    end loop;
  end loop;
end loop;
```

Punti Chiave della Verifica

1. **Handshake Compliance:** Il loop `while` garantisce che il testbench si fermi se la FSM abbassa `tready` (es. durante il reset)
2. **Fase di Flush:** Al termine dei loop, `tvalid` va a 0. Verifichiamo che la FSM attivi il `flush_pipeline` e che il buffer continui a avanzare inserendo zeri (padding) fino allo svuotamento completo

7. Risultati e Conclusioni

Validazione del design e analisi *post-implementation*

Metodologia di Validazione

Per verificare il corretto funzionamento del circuito, abbiamo implementato un flusso di test automatizzato basato su script MATLAB ed Excel

Caso d'uso 1: Filtro Gaussiano

Test con filtro sfocatura (blurring) per verificare il comportamento del circuito

Parametri del Filtro

- Kernel: Matrice simmetrica 3×3
- Normalizzazione: Divisione finale per 16

Col 1	Col 2	Col 3
1	2	1
2	4	2
1	2	1

Risultato Visivo

Confronto tra l'input e l'output elaborato

Caso d'uso 2: Filtro di Sobel (Verticale)

Test con filtro Sobel (vertical edge detection) per analizzare il comportamento con coefficienti negativi e nulli

Parametri del Filtro

- Kernel: Rilevamento dei gradienti verticali
- Normalizzazione: Divisione finale per 4

Col 1	Col 2	Col 3
-1	0	1
-2	0	2
-1	0	1

Risultato Visivo

I bordi verticali della scacchiera vengono evidenziati correttamente

Analisi Post-Implementation e Confronto

L'analisi dei report generati da Vivado rivela differenze architetturali significative tra i due filtri, nonostante il VHDL di base sia identico

Parametri	Filtro Gaussiano	Filtro Sobel (Verticale)	Differenza
WNS (Worst Negative Slack)	2.460 ns	2.812 ns	0.352 ns
LUTs	158	153	-5
Registers	155	159	4
Dynamic power	12 mW	13 mW	1 mW
Static power	105 mW	104 mW	-1 mW
Total On-Chip power	117 mW	117 mW	0 mW

Perché il filtro di Sobel è più efficiente?

L'ottimizzatore (*Synthesizer*) riconosce i coefficienti statici del filtro:

1. **Colonne a Zero:** Nel Sobel, la colonna centrale è composta da `0`, il sintetizzatore elimina fisicamente i moltiplicatori e le linee dati associate
2. **Critical Path Ridotto:** Operandi nulli o più piccoli generano riporti più corti nell'albero CSA, velocizzando il percorso critico
3. **Risparmio Area/Potenza:** La rimozione logica (*pruning*) riduce l'utilizzo di LUT e il conseguente consumo di *potenza statica*

Note: Clock period 10 ns (100 MHz)

Grazie per l'attenzione